Automated-review-rating-system

This project aims to automatically predicts the numerical ratings of the reviews given by the user. It uses Natural Language Processing (NLP), & Machine learning techniques for it.

Project overview and objective

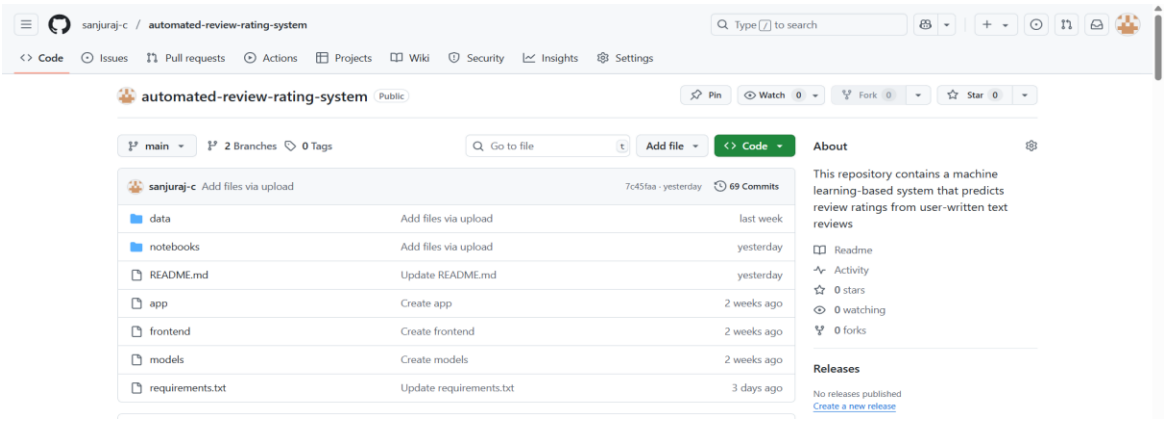
This project aims to build an automated review rating system that analyzes customer reviews and predicts the rating associated with each review. In many real-world scenarios, users provide detailed textual feedback, but numeric ratings may be missing, inconsistent, or biased. The goal is to train a machine learning model that can understand the sentiment and content of a review and accurately classify it into one of the standard rating categories (e.g., 1 to 5 stars).

By processing and analyzing review text using natural language processing techniques, this system helps businesses quickly identify the sentiment of customer feedback, monitor product performance, and make data-driven decisions without manual intervention.

Environmental Setup

The project was implemented using Python for data processing and analysis due to its rich ecosystem of libraries. Development and testing were carried out in Jupyter Notebook, providing an interactive environment for writing and visualizing code outputs. The entire codebase, along with dataset references and updates, was version-controlled using GitHub for collaboration and backup.

GitHub Project Setup:



Dataset description

To build the automated review rating system, data was gathered in two phases from various sources like Kaggle, Google search datasets, Data web scraping using API Key method in which, containing customer reviews along with corresponding ratings. The objective was to retain only the review text and rating fields after cleaning and preprocessing.

Phase 1 Datasets:

1. Amazon Echo Product Reviews (https://www.kaggle.com/datasets/PromptCloudHQ/amazon-echo-dot-2-reviews-dataset)

-This dataset contains reviews specifically for the Amazon Echo Dot 2, collected during September and October 2017. It includes detailed user feedback and metadata related to review authenticity and helpfulness. The complete version of this dataset is available via Data Stock, which contains reviews dating back to 2016.

-Key Fields: Page URL, Title, Review Text, Device Color, User Verified, Review Date, Useful Count, Configuration, Rating, Declaration Text.

2. Amazon Product Reviews (https://www.kaggle.com/datasets/thedevastator/amazon-customer-reviews-with-2013-2019-sentiment)

-This dataset contains reviews for a wide range of Amazon products such as smartphones, laptops, books, and refrigerators. Spanning six years, it also includes an in-house sentiment score based on the review content.

-Key Fields: Category, Review_Header, Review_text, Rating, Own_Rating

3. ChatGPT App Reviews (https://www.kaggle.com/datasets/anandshaw2001/chatgpt-users-reviews)

-This dataset comprises user reviews and ratings for the ChatGPT application. The reviews span several months and include timestamps, making the dataset suitable for temporal sentiment analysis.

-Key Fields: Review Id, Review, Ratings, Review Date.

4. Amazon General Reviews (https://www.kaggle.com/datasets/dilekbarutu/amazon-review)

-A comprehensive dataset containing user reviews for various products sold on Amazon. It includes essential metadata such as user IDs, timestamps, and helpfulness scores.

-Key Fields: reviewerID, asin, reviewerName, helpful, reviewText, overall, summary, unixReviewTime, reviewTime, day_diff

These cleaned datasets were combined to form Cleaned Dataset 1.

Phase 2 Datasets:

1. MakeupAlley Cosmetic Product Reviews (https://www.kaggle.com/datasets/winston56/johnson-johnson-ogx-product-reviews)

-These datasets contain reviews scraped from MakeupAlley and Ulta Beauty. Identifiable information has been removed. While the Ulta dataset includes accurate day-level timestamps, the MakeupAlley dataset includes only month and year.

-Key Fields: Date, Rating, Review, Product

2. Women's Clothing E-commerce Reviews (https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews)

-This dataset contains anonymized customer reviews for women's clothing products. It is suitable for NLP and recommendation system applications due to its structured features and diversity in product types.

-Included customer demographics and feedback information.

-Key Fields: Clothing ID, Age, Title, Review Text, Rating, Recommended IND, Positive Feedback Count, Division Name, Department Name, Class Name

These cleaned datasets were combined to form Cleaned Dataset 2.

Finally, both Cleaned Dataset 1 and Cleaned Dataset 2 were merged into a single dataset used for model training. Only reviews with meaningful content and valid rating labels were preserved.

Preprocessing Steps

After collecting and cleaning the datasets, the following initial preprocessing steps were applied to prepare the review text and rating data for model training:

1. Data Loading and Inspection

Each dataset was loaded using pandas and inspected using functions like .head(), .tail(), .sample(), .shape, .columns, .info(), and .describe() to understand structure and content.

```
#Load
df = pd.read_csv('data.csv')

#View
df.head()          # First 5 rows
df.tail()          # Last 5 rows
df.sample(3)       # Random 3 rows

#Inspect
df.shape           # Rows and columns
df.columns         # Column names
df.info()          # Data types and nulls
df.describe()      # Summary stats
```

2. Handling Missing Values

-Columns that had a large number of missing values or were irrelevant to the analysis were dropped.
-Rows with missing entries in the review text or rating columns were removed to maintain data quality.

```
df.isnull().sum()                  # Count nulls
df.dropna(inplace=True)            # Drop rows with any nulls
df['x'].fillna(df['x'].mean(), inplace=True)  # Fill with mean
```

3. Dropping Irrelevant Columns

-Only the review text and rating columns were retained from each dataset.
-Other metadata like dates, user details, product configuration, etc., were considered unnecessary and discarded.

```
df.drop('x', axis=1, inplace=True)        # Drop column
```

4. Duplicate Removal
-All datasets were checked for duplicate rows using .duplicated().
-Any duplicate records found (if any) were removed to avoid data redundancy.

```
df.duplicated().sum()    #finding duplicates
df.drop_duplicates(inplace=True)    #removing duplicates
```

5. Data Merging

-Cleaned datasets from Phase 1 were combined into a single dataset.
-Similarly, Phase 2 datasets were cleaned and merged.
-Finally, both cleaned datasets were concatenated to form one unified dataset.

```
df_combined = pd.concat([df1, df2], axis=0)    # Stack vertically
```

Review Text Processing

6. Emoji and Special Character Removal

-Emojis, symbols, and special characters were removed using regular expressions to focus only on meaningful textual content.

```
# Remove emojis
text = re.sub(r'[^\w\s,]', '', text.encode('ascii', 'ignore').decode('ascii'))

# Remove special characters and digits (keep letters and basic punctuation if needed)
text = re.sub(r'[^A-Za-z\s]', '', text)
```

7. Stop Word Removal

-Common stop words (e.g., “the”, “is”, “and”) were removed using NLTK to reduce noise and retain only informative words.

```
#defining stopword removal function
stop_words = set(stopwords.words('english'))
punctuations = string.punctuation

def remove_stopwords(text):
    # Lowercase
    text = text.lower()
    # Remove punctuation
    text = ''.join([char for char in text if char not in punctuations])
    # Tokenize and remove stopwords
    words = text.split()
    filtered = [word for word in words if word not in stop_words]
    return ' '.join(filtered)
```

-Stopwords used:

-From NLTK
-stop word count= 198

['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any', 'are', 'aren', "aren't", 'as', 'at', 'be', 'because', 'been', 'before', 'being', 'below', 'between', 'both', 'but', 'by', 'can', 'couldn', "couldn't", 'd', 'did', 'didn', "didn't", 'do', 'does', 'doesn', "doesn't", 'doing', 'don', "don't", 'down', 'during', 'each', 'few', 'for', 'from', 'further', 'had', 'hadn', "hadn't", 'has', 'hasn', "hasn't", 'have', 'haven', "haven't", 'having', 'he', "he'd", "he'll", "he's", 'her', 'here', 'hers', 'herself', 'him', 'himself', 'his', 'how', 'i', "i'd", "i'll", "i'm", "i've", 'if', 'in', 'into', 'is', 'isn', "isn't", 'it', "it'd", "it'll", "it's", 'its', 'itself', 'just', 'll', 'm', 'ma', 'me', 'mightn', "mightn't", 'more', 'most', 'mustn', "mustn't", 'my', 'myself', 'needn', "needn't", 'no', 'nor', 'not', 'now', 'o', 'of', 'off', 'on', 'once', 'only', 'or', 'other', 'our', 'ours', 'ourselves', 'out', 'over', 'own', 're', 's', 'same', 'shan', "shan't", 'she', "she'd", "she'll", "she's", 'should', "should've", 'shouldn', "shouldn't", 'so', 'some', 'such', 't', 'than', 'that', "that'll", 'the', 'their', 'theirs', 'them', 'themselves', 'then', 'there', 'these', 'they', "they'd", "they'll", "they're", "they've", 'this', 'those', 'through', 'to', 'too', 'under', 'until', 'up', 've', 'very', 'was', 'wasn', "wasn't", 'we', "we'd", "we'll", "we're", "we've", 'were', 'weren', "weren't", 'what', 'when', 'where', 'which', 'while', 'who', 'whom', 'why', 'will', 'with', 'won', "won't", 'wouldn', "wouldn't", 'y', 'you', "you'd", "you'll", "you're", "you've", 'your', 'yours', 'yourself', 'yourselves']

8. Lemmatization

-Words were reduced to their root forms (e.g., “running” → “run”) using WordNetLemmatizer, improving consistency across similar word forms.

```
#defining lemmatization function
lemmatizer = WordNetLemmatizer()

def lemmatize_text(text):
    words = text.split()
    lemmatized = [lemmatizer.lemmatize(word) for word in words]
    return ' '.join(lemmatized)
```

-**Why Lemmatization Over Stemming?**

Lemmatization was chosen over stemming because it provides more accurate and meaningful word forms by reducing words to their dictionary base form (lemma), whereas stemming often results in incomplete or non-existent words. For example, "running" becomes "run" through lemmatization, but stemming might reduce it to "runn". This helps retain better context and grammatical correctness, which is especially important for tasks involving sentiment analysis and text understanding.

9. Filtering Out Short and Long Reviews

-Reviews with fewer than 3 words or more than 200 words were removed to retain only meaningful and focused feedback.

```
# Keep only reviews that have word count between 3 and 200

filtered_df = df[df['Review'].apply(lambda x: 3 <= len(x.split()) <= 200)].reset_index(drop=True)
```

10. Conflicting Review Removal

-Reviews that did not align with their ratings (e.g., positive text with a negative rating or vice versa) were manually identified and removed.

```
#checking conflicting reviews
from textblob import TextBlob

# Get sentiment polarity for each review (-1 = negative, 1 = positive)
df['Polarity'] = df['Review'].apply(lambda x: TextBlob(x).sentiment.polarity)

# Optional: classify sentiment direction
def classify_sentiment(p):
    if p > 0.2:
        return 'positive'
    elif p < -0.2:
        return 'negative'
    else:
        return 'neutral'
df['Sentiment'] = df['Polarity'].apply(classify_sentiment)

# Define conflict conditions
conflict_mask = (
    ((df['Rating'] >= 4) & (df['Sentiment'] == 'negative')) |
    ((df['Rating'] <= 2) & (df['Sentiment'] == 'positive'))
)

# Get conflicting reviews for optional inspection
conflicting_reviews = df[conflict_mask]

# Remove conflicting reviews
cleaned_df = df[~conflict_mask].reset_index(drop=True)
```

These preprocessing steps ensured a high-quality, cleaned dataset .

Visualizations Used

To better understand the data and guide modeling decisions, several visualizations were created during the exploratory data analysis (EDA) phase:

1. Histogram Plots of Rating Distributions

-Visualized how ratings were distributed across the dataset.

-Helped identify class imbalances (e.g., more 5-star reviews than 1-star).

2. Countplots (Per Rating Class)

-Displayed the number of reviews for each rating class using count plots.

-Useful for comparing rating frequency across different datasets.

3. Pie Charts of Overall Sentiment Distribution

-Ratings were grouped into broader sentiment categories like Positive, Neutral, and Negative.

-Pie charts helped visualize the proportion of each sentiment group in the dataset.

Before and After Cleaning/Balancing Comparisons

-Visualizations were also used to compare the dataset before and after cleaning and balancing. This helped in understanding how much the data was affected by preprocessing steps, especially in terms of class distribution. It clearly showed how the ratings became more evenly distributed after balancing, making the dataset suitable for training.

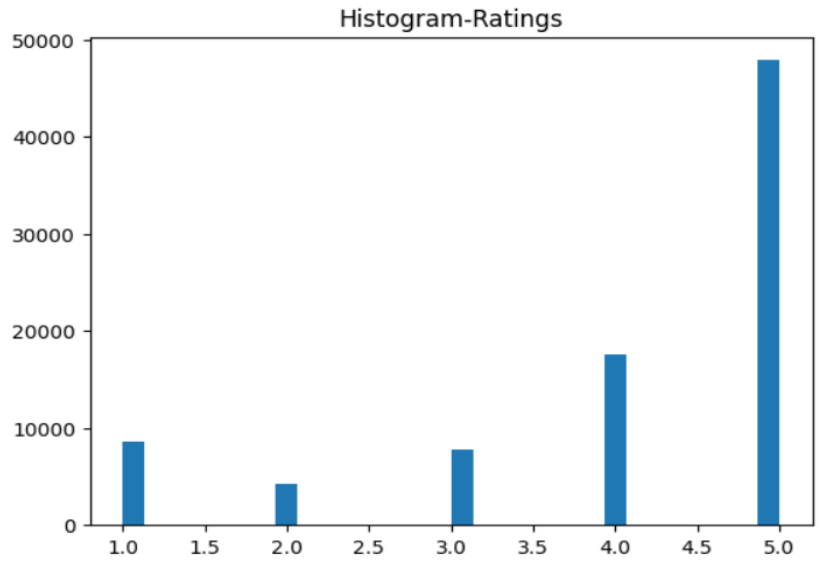
Final Combined Dataset

-It is the combination of both cleaned datasets of two phases.
-Total 6 different datasets are included in it.
-It has 86137 rows and 2 columns of 'Review Text' & 'Rating'.

Done some visualization on the final combined data:

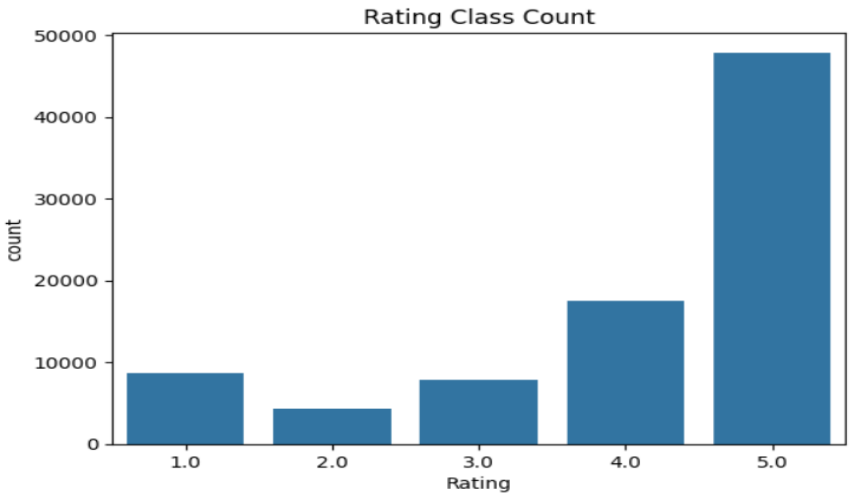
1.Histogram

```
plt.hist(final_combined_df['Rating'],bins=30)
plt.title('Histogram-Ratings')
plt.show()
```



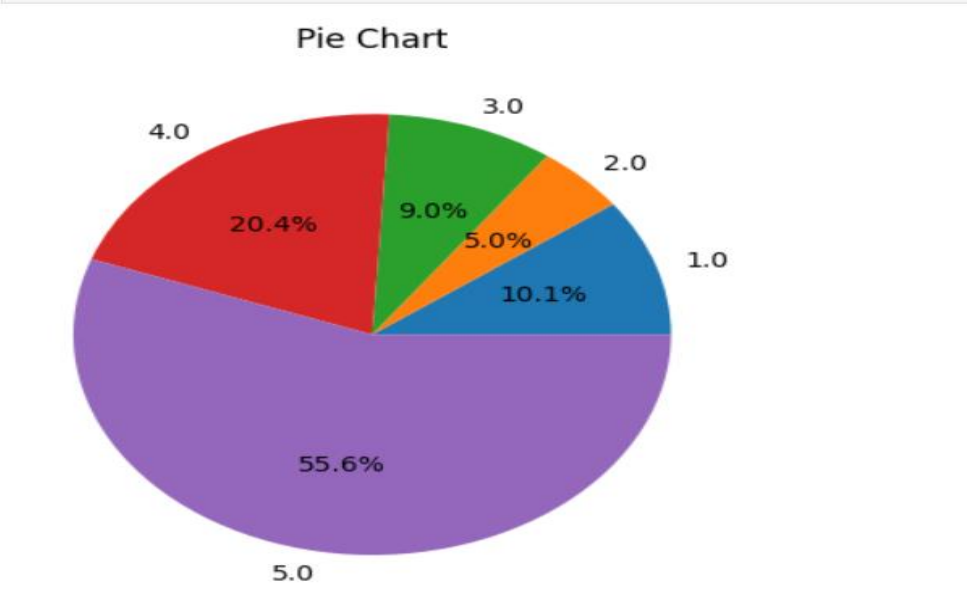
2.Count plot

```
sns.countplot(x='Rating', data=df2)
plt.title("Rating Class Count")
plt.show()
```



3. Pie Chart

```
rating_counts = df2['Rating'].value_counts().sort_index()
plt.pie(rating_counts, labels=rating_counts.index, autopct='%1.1f%%')
plt.title("Pie Chart")
plt.show()
```



From the above visualizations, we can understand the rating data distribution. Rating 5 is higher in data and rating 2 is the lowest one.

Imbalanced Dataset

-Imbalanced dataset created from the final cleaned data excluding the balanced data content from it.
-By loading it and again checked for any null values or duplicates for further clarificiation.
-Created the unbalanced dataset with rating data distribution of 1=10%, 2=15%, 3=25%, 4=30% & 5=20%.

```
# Remove balanced data from cleaned_combined_df
remaining_df = pd.concat([cleaned_df, balanced_df]).drop_duplicates(keep=False)

target_size = 10000
distribution = {
   1.0: int(target_size * 0.10),  # 10%
   2.0: int(target_size * 0.15),  # 15%
   3.0: int(target_size * 0.25),  # 25%
   4.0: int(target_size * 0.30),  # 30%
   5.0: int(target_size * 0.20),  # 20%
}

# Step: Sample based on target distribution
unbalanced_samples = []

for rating, count in distribution.items():
   subset = remaining_df[remaining_df['Rating'] == rating]
```

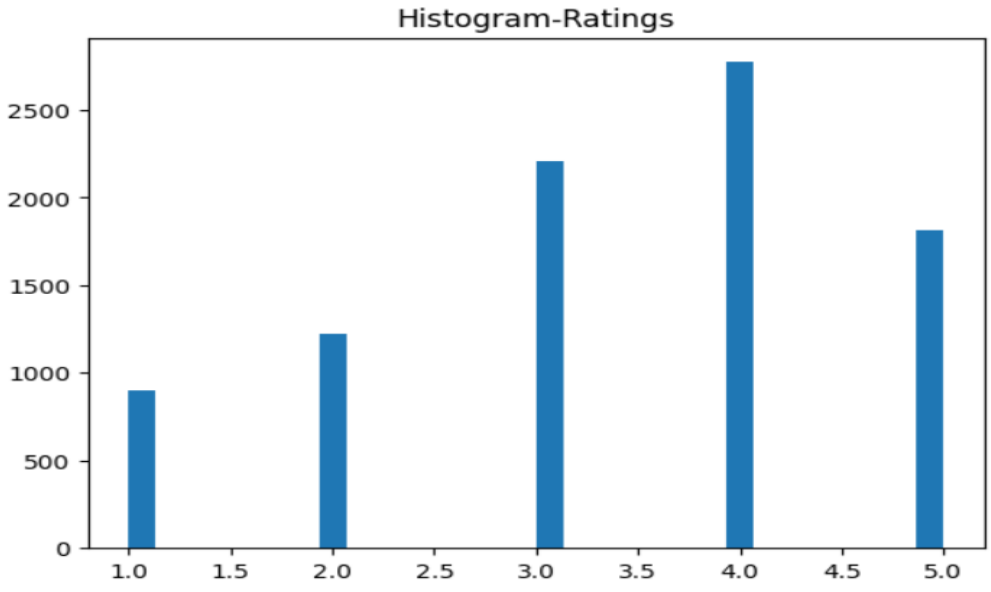
```
    available = len(subset)
    if available >= count:
        sample = subset.sample(n=count, random_state=42)
    else:
        print(f" Only {available} samples available for Rating {rating}, requested {count}")
        sample = subset  # take all available
    unbalanced_samples.append(sample)

# Combine into final unbalanced dataset
unbalanced_df = pd.concat(unbalanced_samples).reset_index(drop=True)
```

Done some visualization on the Imbalanced data:

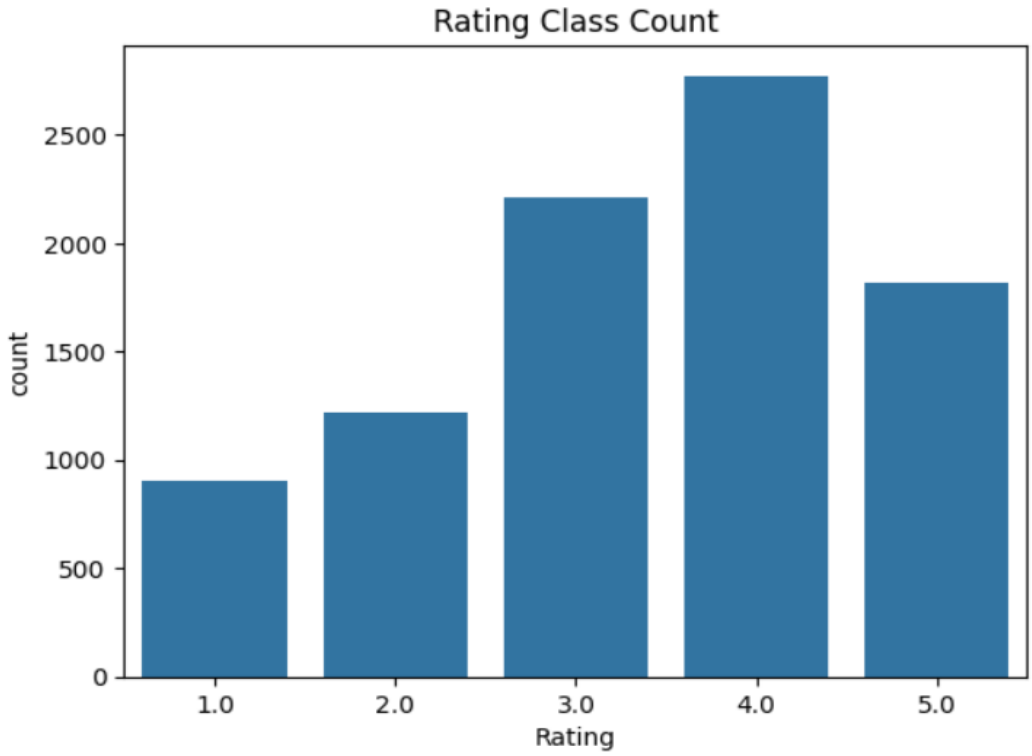
1.Histogram

```
plt.hist(filtered_df['Rating'],bins=30)
plt.title('Histogram-Ratings')
plt.show()
```



2.Countplot

```
sns.countplot(x='Rating', data=filtered_df)
plt.title("Rating Class Count")
plt.show()
```



Sample Reviews Per Rating in Balanced Data:

```
for rating in sorted(imbalanced_df_final['Rating'].unique()):
   print(f"\n Rating: {int(rating)}")
   samples = imbalanced_df_final[imbalanced_df_final['Rating'] == rating].sample(3,
random_state=42)
   for i, review in enumerate(samples['Review'].values, 1):
        print(f"{i}. {review}\n")
```

output:

Rating: 1
1. overall seemed like decent phone point time show network availableemergency call another phone show almost full bar network tried rebooting airplane mode reset network nothing work sure firmware issue o issue make using phone precarious experience expected better

2. one month used phone battery life good video camera quality good hitting problem charger front camera good

3. card worked great first ive never problem sandisk past bought card galaxy back october dead december im sorely disappointed purchase maybe faulty card experience terrible get one star

Rating: 2
1. disappointed ordered dress large big returned asked exchange size medium apparently available medium charged fee return shopping immediately went website medium available pay another shipping seems bit unreasonable really liked dress

2. problem processing suddenly stop operation n go sleep mode keypad function erroneously automatically start erasing whatever typevery strange

3. overall device fine camera quality bad worse old device redmi selfie camera like mp camera another samsung phone photo people redmi look much artificial colour look much different real front rear camera camera device disappointed otherwise device fine interest camera centric device buy else go option important point phone bit o instead bit search google

Rating: 3

1. love fabric pattern dress much decided keep got mega sale would never paid full price anything close armhole enormous showing bra particularly large chested c button stretched gaping even huge armhole cute red sweater coat wear fall could recommend poor cut design pro great pattern fabric pocket true

2. loud enough say bluetooth speaker

3. wine color beautiful idea knot front different back good one however fabric thin back quite see knit show every single bodily imperfection quintessential tall thin dont mind showing bra go otherwise suggest skipping one

Rating: 4
1. good battery life minute charge sleep monitor distance calorie sensor working well see buy st time dosent heart rate sensor thats fault band another feature good

2. sound quality top notch bass deep get perfect low build quality mark wire thin tangle free use carefully

3. redmi good looking phone good battery backup screen resolution could better phone costing k guess cannot expect better

Rating: 5
1. amazing product amazon want say product inch led android smart

2. wish slightly cheaper could buy love little guy kidding alexa girl ton fun always coming new feature
3. osm must buy

Min/Max Review Length and Counts in Balanced Data

```
for rating in sorted(imbalanced_df_final['Rating'].unique()):
   # Filter data by rating
   subset = imbalanced_df_final[imbalanced_df_final['Rating'] == rating]

   # Compute review lengths (word count)
   lengths = subset['Review'].apply(lambda x: len(x.split()))

   min_len = lengths.min()
   max_len = lengths.max()

    min_count = (lengths == min_len).sum()
   max_count = (lengths == max_len).sum()

   print(f"\n Rating: {int(rating)}")
   print(f"   Min review length: {min_len} words (Count: {min_count})")
   print(f"   Max review length: {max_len} words (Count: {max_count})")
```

Output:

Rating: 1
Min review length: 3 words (Count: 57)
Max review length: 165 words (Count: 1)

Rating: 2
Min review length: 3 words (Count: 51)
Max review length: 174 words (Count: 1)

Rating: 3
Min review length: 3 words (Count: 93)
Max review length: 165 words (Count: 2)

Rating: 4
Min review length: 3 words (Count: 126)
Max review length: 200 words (Count: 1)

Rating: 5
Min review length: 3 words (Count: 120)
Max review length: 194 words (Count: 1)

Train-Test-split data for Training

To evaluate the model's performance effectively, the dataset was split into two parts: 80% for training and 20% for testing. The training set was used to fit the model, while the test set helped in assessing its performance on unseen data. This split ensures better generalization and avoids overfitting.

```
from sklearn.model_selection import train_test_split

# Features and labels
X = imbalanced_df_final['Review']
y = imbalanced_df_final['Rating']

# Train-test split (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, stratify=y,
random_state=42)
```

-Why Stratified Splitting?

Stratified splitting was used to ensure that the distribution of classes (ratings) remains consistent in both the training and testing sets. This is especially important when dealing with imbalanced datasets, as it helps maintain the proportion of each class across splits, leading to more reliable evaluation and better generalization of the model.

TF-IDF Vectorization

Since machine learning models cannot directly process raw text, vectorization was used to convert textual data into numerical format. In this project, the TF-IDF (Term Frequency–Inverse Document Frequency) method was applied, which transforms the text into a matrix that reflects how important a word is to a document in a collection. This approach not only captures the frequency of terms but also reduces the impact of commonly occurring words, helping the model focus on more informative terms for better predictions.

Term Definitions:

Term Frequency (TF):
This shows how often a word appears in a document.

$$TF = \frac{Number\ of\ times\ the\ word\ appears\ in\ the\ document}{Total\ number\ of\ words\ in\ the\ document}$$

Inverse Document Frequency (IDF):
This tells how unique or rare a word is across all documents.

$$IDF = log\left(\frac{Total\ number\ of\ documents}{1 + number\ of\ documents\ containing\ the\ word}\right)$$

TF-IDF Score:
This is the final value used to represent each word. It is the product of TF and IDF.

$$TF - IDF = TF \times IDF$$

TF-IDF helps to give more importance to words that are unique and meaningful, and reduces the weight of common words.

```
from sklearn.feature_extraction.text import TfidfVectorizer

# Initialize TF-IDF vectorize
tfidf = TfidfVectorizer(max_features=5000)  # or use None for all features

# Fit on training data and transform both train and test
X_train_tfidf = tfidf.fit_transform(X_train)
X_test_tfidf = tfidf.transform(X_test)
```

ML algorithm used- Random Forest

https://github.com/sanjuraj-c/automated-review-rating-system/blob/main/ML%20Algorithm%20used%20for%20the%20model-imbalanced%20data.pdf

Model Choice and Reason

-I selected Random Forest for this phase because it is well-suited for handling imbalanced datasets and can capture non-linear relationships in the data.
-Unlike simpler models such as Logistic Regression, Random Forest uses an ensemble of decision trees, which can provide more robust predictions for complex text classification tasks.
-It also works well with high-dimensional data generated from TF-IDF vectorization.

Model Training Logic

1.Data Preparation – The imbalanced dataset was split into training and testing sets while maintaining class distribution (stratified split).

2.Model Initialization – I used a RandomForestClassifier with:

-n_estimators=200 (number of trees)
-random_state=42 (reproducibility)

3.Model Training – The model was trained on the TF-IDF vectorized review text (X_train) and corresponding ratings (y_train).

4.Prediction – The trained model was used to predict ratings for the test set (y_test).

5.Evaluation – Performance was measured using accuracy, precision, recall, F1-score, and a confusion matrix to see where the model performs well or struggles.

Evaluation results and interpretation

1.Accuracy: 0.4504 (~45%)

2. Precision/Recall/F1-score:

Classification Report:

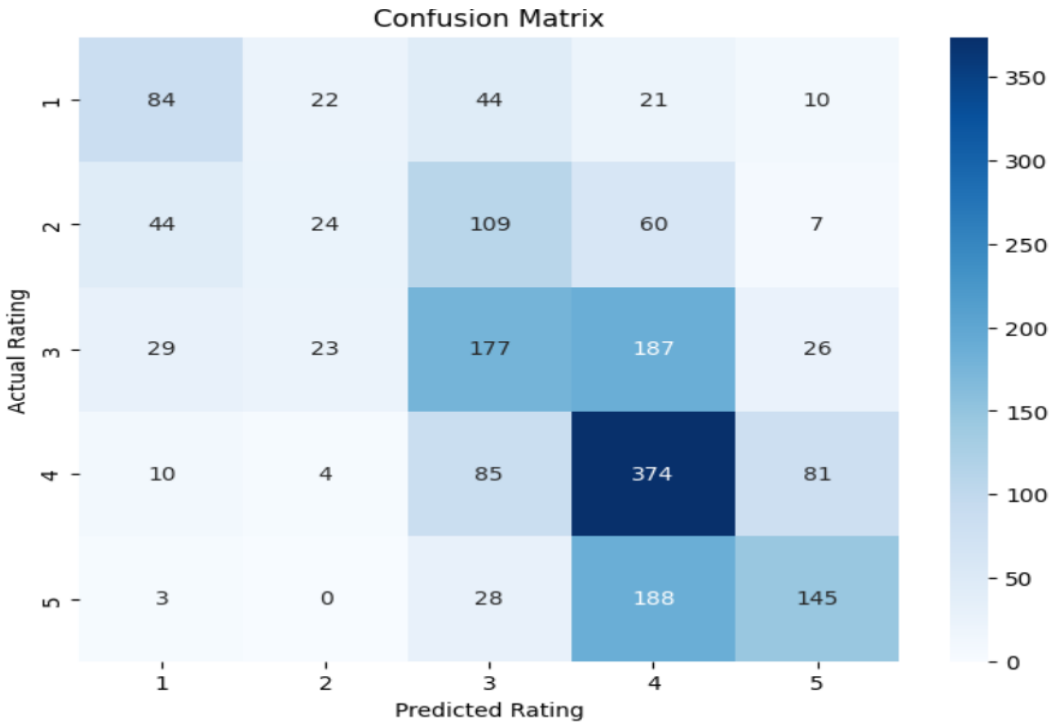
	precision	recall	f1-score	support
1.0	0.494	0.464	0.479	181
2.0	0.329	0.098	0.151	244
3.0	0.400	0.400	0.400	442
4.0	0.451	0.675	0.540	554
5.0	0.539	0.398	0.458	364
accuracy			0.450	1785
macro avg	0.442	0.407	0.406	1785
weighted avg	0.444	0.450	0.429	1785

-Class 4 (Rating 4) had the highest recall (0.675), meaning it correctly identified most actual 4-star reviews.

-Class 2 (Rating 2) had the lowest recall (0.098), indicating difficulty in predicting this rating.

-Class 5 had the highest precision (0.539), meaning when the model predicts a 5-star review, it is correct most of the time.

3.Confusion matrix



From the confusion matrix:

-Many 3-star reviews were confused with 4-star reviews, and 1-star reviews were sometimes misclassified as 3-star.

-Some overlap exists between neighboring rating classes (e.g., 4 and 5), which is common in review sentiment classification.